

BÀI GIẢI 3 CÂU

Câu 1. Tìm bug trong hàm union() của Quick-Find

Ta cần tìm một testcase làm cho hai phần tử i và j cùng tập hợp nhưng $\text{find}(i)$ và $\text{find}(j)$ trả về khác nhau.

Chọn $n = 3$, thực hiện các phép toán:

```
union(0, 1)
union(0, 2)
```

Ban đầu: $\text{leader} = [0, 1, 2]$.

Sau $\text{union}(0,1)$: $\text{leader} = [1, 1, 2]$.

Sau $\text{union}(0,2)$: vì $\text{leader}[0] = 1$ nên các phần tử có leader bằng 1 được đổi thành $\text{leader}[2] = 2$. Ta được $\text{leader} = [2, 2, 2]$. Testcase này vẫn đúng.

Để làm lộ bug, chọn $n = 4$ và thực hiện:

```
union(0, 1)
union(2, 3)
union(0, 2)
```

Sau $\text{union}(0,1)$: $[1, 1, 2, 3]$.

Sau $\text{union}(2,3)$: $[1, 1, 3, 3]$.

Sau $\text{union}(0,2)$: $\text{leader}[0] = 1$ và $\text{leader}[2] = 3$. Vì vậy các phần tử 0,1 được đổi từ 1 thành 3. Kết quả $[3,3,3,3]$. Testcase này vẫn đúng, nên ta cần chú ý bug thực sự nằm ở việc dùng $\text{leader}[p]$ thay đổi trong vòng lặp.

Một testcase làm sai là $n = 4$, $\text{union}(0,1)$, sau đó $\text{union}(0,2)$ rồi $\text{union}(0,3)$ nếu phép thay đổi $\text{leader}[p]$ xảy ra trong lúc duyệt. Trong code đã cho, $\text{leader}[p]$ có thể bị thay đổi khi $i = p$, làm điều kiện ở các vòng sau thay đổi. Cách sửa đơn giản là lưu $\text{rootP} = \text{leader}[p]$ trước vòng lặp.

```
public void union(int p, int q) {
    int rootP = leader[p];
    int rootQ = leader[q];
    for (int i = 0; i < leader.length; i++)
        if (leader[i] == rootP)
            leader[i] = rootQ;
}
```

Câu 2. Quick-Find, Quick-Union và Weighted Quick-Union

Thực hiện chuỗi: $\text{union}(0,1), \text{union}(0,2), \dots, \text{union}(0,n-1)$.

(a) Số thành phần liên thông

Ban đầu có n thành phần. Mỗi phép union làm giảm số thành phần đi 1. Có $n - 1$ phép union nên cuối cùng còn: 1 thành phần liên thông.

(b) Quick-Find

Trong lần union(0,1), có $n - 1$ phần tử cần cập nhật. Trong mỗi lần tiếp theo union(0,k), có 2 phần tử (0 và 1) cần cập nhật. Vì vậy số lần cập nhật là:

$$(n - 1) + 2(n - 2) = 3n - 5 = \Theta(n)$$

Sau chuỗi phép toán, find(0) có thời gian $\Theta(1)$.

(c) Quick-Union

Mỗi phép union(0,k) đặt $\text{parent}[k] = 0$. Do đó tất cả các phần tử $1, 2, \dots, n-1$ đều trở trực tiếp về 0. Khi gọi find(0), chỉ cần kiểm tra $\text{parent}[0] = 0$, nên số lần truy cập mảng là $\Theta(1)$.

(d) Weighted Quick-Union

Cây chứa phần tử 0 luôn là cây lớn hơn sau mỗi phép union. Vì vậy các phần tử mới được nối vào gốc 0. Cây cuối cùng có chiều cao 1. Do đó find(0) cũng chỉ mất $\Theta(1)$.

Câu 3. Faster 3-Sum

Mảng đã được sắp xếp. Ta dùng hai con trỏ left và right thay vì tìm kiếm nhị phân cho từng cặp.

```
public static int count(int[] a) {
    int n = a.length;
    int count = 0;

    for (int i = 0; i < n - 2; i++) {
        int left = i + 1;
        int right = n - 1;

        while (left < right) {
            int sum = a[i] + a[left] + a[right];

            if (sum == 0) {
                count++;
                left++;
                right--;
            } else if (sum < 0) {
                left++;
            } else {
                right--;
            }
        }
    }

    return count;
}
```

Giải thích: với mỗi i , left bắt đầu ở $i + 1$ và right bắt đầu ở cuối mảng. Nếu tổng nhỏ hơn 0 thì tăng left để tổng lớn hơn. Nếu tổng lớn hơn 0 thì giảm right để tổng nhỏ

hơn. Nếu tổng bằng 0 thì tìm được một bộ ba.

Với mỗi i , vòng while chạy tối đa $O(n)$ lần. Vòng for có $O(n)$ giá trị i . Vì vậy độ phức tạp thời gian là $\Theta(n^2)$, tốt hơn thuật toán 3-Sum dùng tìm kiếm nhị phân có độ phức tạp $O(n^2 \log n)$.